Phase 3: Global State (Zustand) + 60 FPS Batching

What we built

- A **global store** (`marketStore`, built with Zustand) that holds the live tickers, the selected symbol and the connection status
- A `FrameBatcher` that collects WebSocket ticks and applies them **once per animation frame** with `requestAnimationFrame`
- **Fine-grained subscriptions**: each component reads only its own slice of the store, so a BTC tick re-renders only the BTC row
- **Green/red flash** on every price change, in the watchlist and on the big price
- A **"msg/s → renders/s"** counter in the header, so you can *see* the batching work

Files and what each one does

File	Role
`src/store/marketStore.ts`	**The store.** State + actions in one place. `applyTickers` applies a whole batch in one update
`src/services/FrameBatcher.ts`	Buffers updates in a Map and flushes them once per frame with rAF
`src/hooks/useMarketFeed.ts`	Replaces `useLiveTickers`: socket → batcher → store. Returns nothing
`src/components/FlashingPrice.tsx`	A price that flashes green/red when it changes
`src/components/DocumentTitle.tsx`	Tab title, moved out of App so App never re-renders
`src/components/FeedStats.tsx`	The msg/s → renders/s counter

Data flow: before vs after



Concepts (theory only)

1. Prop drilling, and why global state solves it

Prop drilling: passing data through components that don't use it, just so it reaches a grandchild. In Phase 2, `tickers` went App → Watchlist → WatchlistRow, and `onSimulateDrop` went App → Header → ConnectionBadge. Problems: noisy code, and the **owner** (App) re-renders on every change, which re-renders the whole tree under it. **Global store**: the state lives *outside* the component tree. Any component can read it directly, no matter how deep it is.

2. Zustand in one sentence

`create()` gives you a **hook**. You call it with a **selector**, `useMarketStore(s => s.tickers.BTCUSDT)`, and the component re-renders **only when that selected value changes** (compared with `Object.is`, i.e. same reference / same number). - State and the actions that change it live in the same object - No `<Provider>` wrapper is needed - `useMarketStore.getState()` reads the store *without* subscribing. It's useful in non-React code, like our feed hook.

3. Redux Toolkit, for comparison

	Redux Toolkit	**Zustand**
Setup	`configureStore` + `createSlice` + `<Provider>`	one `create()` call
Changing state	`dispatch(action)` → **reducer** computes new state	call the action function → `set()`
Reading	`useSelector(s => ...)`	`useStore(s => ...)`
Strengths	strict structure, DevTools time-travel, middleware, huge ecosystem, good for big teams	tiny, little boilerplate, fast to write

Vocabulary: *store* = the single state object · *slice* = one feature's part of it · *action* = an object describing "what happened" (`{type: 'tickers/updated', payload}`) · *reducer* = a pure function `(state, action) => newState`. **Same core ideas in both:** a single source of truth, immutable updates, and selectors. **Why Zustand here:** a trading UI updates very often, and Zustand's selector-based subscriptions are simple and cheap. Redux Toolkit would work too (it's on the resume as an alternative). It's just more ceremony.

4. Selectors and avoiding re-renders

- A component re-renders when the value its selector returns is **different** from last time.
- `applyTickers` copies the outer `tickers` object, but **untouched tickers keep their old object**. So `s.tickers.ETHUSDT` returns the *same reference*, and the ETH row skips its re-render.
- `s.selectedSymbol === item.symbol` returns a **boolean**, so when the selection changes only 2 rows re-render (the old one and the new one).

- ⚠ **Pitfall:** a selector that builds a *new* object or array every time (`s => ({a: s.a, b: s.b})`) always looks "changed", so the component re-renders on *every* store update. Select primitives or existing objects, or use Zustand's `useShallow`.
- **Subscribe low in the tree.** Whoever subscribes re-renders *with all its children*. That's why the tab-title logic moved out of App into `<DocumentTitle />`.

5. `React.memo`

`memo(Component)` = "skip re-rendering me if my **props** are shallow-equal to last time". `WatchlistRow` is memoized, so typing in the search box (which re-renders `Watchlist`) doesn't re-render every row. It doesn't block re-renders from the component's **own** store subscription, and that's what we want.

6. The browser render loop: 60 FPS = 16.6 ms

A typical screen refreshes **60 times a second** → the browser has **~16.6 ms per frame** to run JavaScript, recalculate styles and layout, and paint. If JS takes longer than that, frames are **dropped** → visible stutter ("jank"). Key insight: **updating the UI more often than the screen can paint is pure waste.** 100 updates/sec on a 60 Hz screen means at least 40 of them are never seen.

7. `requestAnimationFrame` (rAF)

`requestAnimationFrame(cb)` runs `cb` **right before the browser paints the next frame**. Our `FrameBatcher`: 1. `push(key, value)` → store it in a Map; if no frame is booked, book one with rAF 2. Next frame → take the whole Map, hand it to `applyTickers` → **one** store update → **one** React render pass 3. Start a fresh Map for the next frame

Coalescing: the Map is keyed by symbol, so 3 BTC ticks inside one frame become just the **latest** one. **Bonus:** rAF **pauses in background tabs**. The buffer then just holds the latest value per symbol (it can't grow forever), and everything catches up in one update when you come back.

Why not `setTimeout(16)` or debounce? rAF is synced to the *actual* display refresh (60, 120 or 144 Hz) and pauses when hidden. A timer drifts out of sync with paints. Debounce would *delay* updates while ticks keep arriving, which is bad for prices.

8. React 18+ automatic batching (don't confuse the two)

React already merges several `setState` calls that happen **in the same event/tick** into one render. But WebSocket messages arrive in **separate** events, so React can't merge them. Our rAF batching merges updates **across** events, up to one frame's worth.

9. The flash effect: restarting a CSS animation with `key`

A CSS animation plays when an element **appears**. `<span key={price}>` makes React treat each new price as a **new element**: it unmounts the old one and mounts a fresh one, so the animation replays. Same price → same key → no flash. The direction (`move: 'up' | 'down'`) is computed in the store by comparing the new price with the old one.

Try it yourself

1. Watch the header: **“8 msg/s → 7 renders/s”**. Renders ≤ messages, because ticks that land in the same frame share one render. With *many* streams (the order book in Phase 6 sends 10 updates/sec per symbol), messages could be in the hundreds, and renders still stay ≤ 60.
2. Watch prices **flash** green/red as they tick.
3. **React DevTools** (Chrome extension) → ⚙ → “Highlight updates when components render”. Only the rows that ticked light up, not the whole page. (In Phase 2 the whole page flashed on every tick.)
4. Switch to another tab for 30s and come back. Prices jump straight to the latest values (rAF paused while hidden).
5. Type in the search box: rows filter, and memo means the remaining rows don’t re-render.

Resume mapping

> “managed real-time state with **Zustand** (evaluated against **Redux Toolkit**)” → `marketStore.ts` + the comparison table above “batched high-frequency ticks with **requestAnimationFrame** to sustain **60 FPS**” → `FrameBatcher.ts` + `useMarketFeed.ts` “live watchlist” → `WatchlistRow` with per-row selectors and the flash

Interview Qs

1. **Why did you add a global store?** To stop prop drilling, and so that every component subscribes only to the slice it needs. Price ticks no longer re-render the whole tree.
2. **Zustand vs Redux Toolkit?** Same ideas (single store, immutable updates, selectors). Redux adds actions/reducers/Provider, which gives structure and great DevTools. Zustand is a single `create()` with very little boilerplate. For a small real-time dashboard I picked Zustand, and I’d choose Redux Toolkit for a large team that needs strict conventions.
3. **How do selectors prevent re-renders?** The component re-renders only when the selector’s result changes by reference/value. I keep unchanged tickers as the same object, so other rows’ selectors return the same thing.
4. **What’s a classic selector bug?** Returning a new object or array each time → it’s “changed” on every update → re-render storm (in Zustand v5 it can even cause an infinite loop). The fix: select primitives, or use `useShallow`.
5. **What does requestAnimationFrame do, and why use it here?** It runs a callback just before the next paint. I buffer ticks in a Map and flush once per frame, so state updates can’t exceed the refresh rate, and multiple ticks for one symbol coalesce into the latest one.
6. **Why 16.6 ms?** 1000 ms / 60 frames. That’s the budget for JS + layout + paint per frame at 60 Hz.
7. **rAF vs setTimeout vs debounce?** rAF is aligned with real paints and pauses in hidden tabs. setTimeout isn’t synced to paints. Debounce delays updates during continuous streams, so prices would freeze.

8. **Doesn’t React already batch?** Only updates within the same event tick. WebSocket messages are separate events, so cross-event batching has to be done by us.
9. **What does `React.memo` do, and when doesn’t it help?** It skips a re-render if the props are shallow-equal. It doesn’t help when props are new objects/functions each render (that’s what `useMemo` / `useCallback` are for), and it doesn’t stop re-renders from the component’s own state or subscriptions.